elacotic

Efficient, portabLe And Secure orchesTration for reliable servICes

TrustMee: Self-Verifying Remote Attestation Evidence

2026-04-07

Parsa Sadri Sinaki[1], Zainab Ahmad[2,1], Wentao Xie[2,1], Merlijn Sebrechts[3], Jimmy Kjällman[2], **Lachlan J. Gunn**[1]

[1]Aalto University [2]Ericsson Finland [3]Ghent University + imec

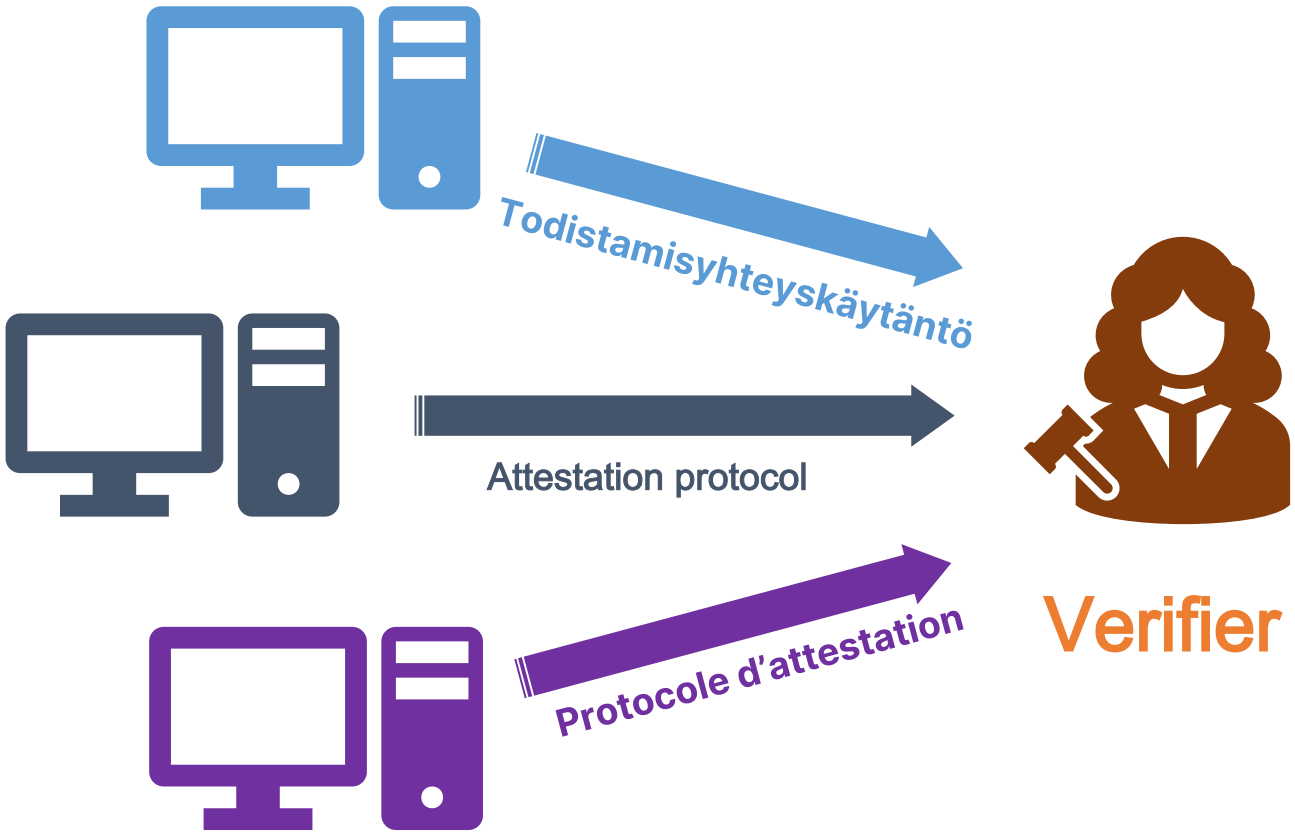
Co-funded by The European Union

6GSNS

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or SNS-JU. Neither the European Union nor the granting authority can be held responsible for them.

The problem

elacotic



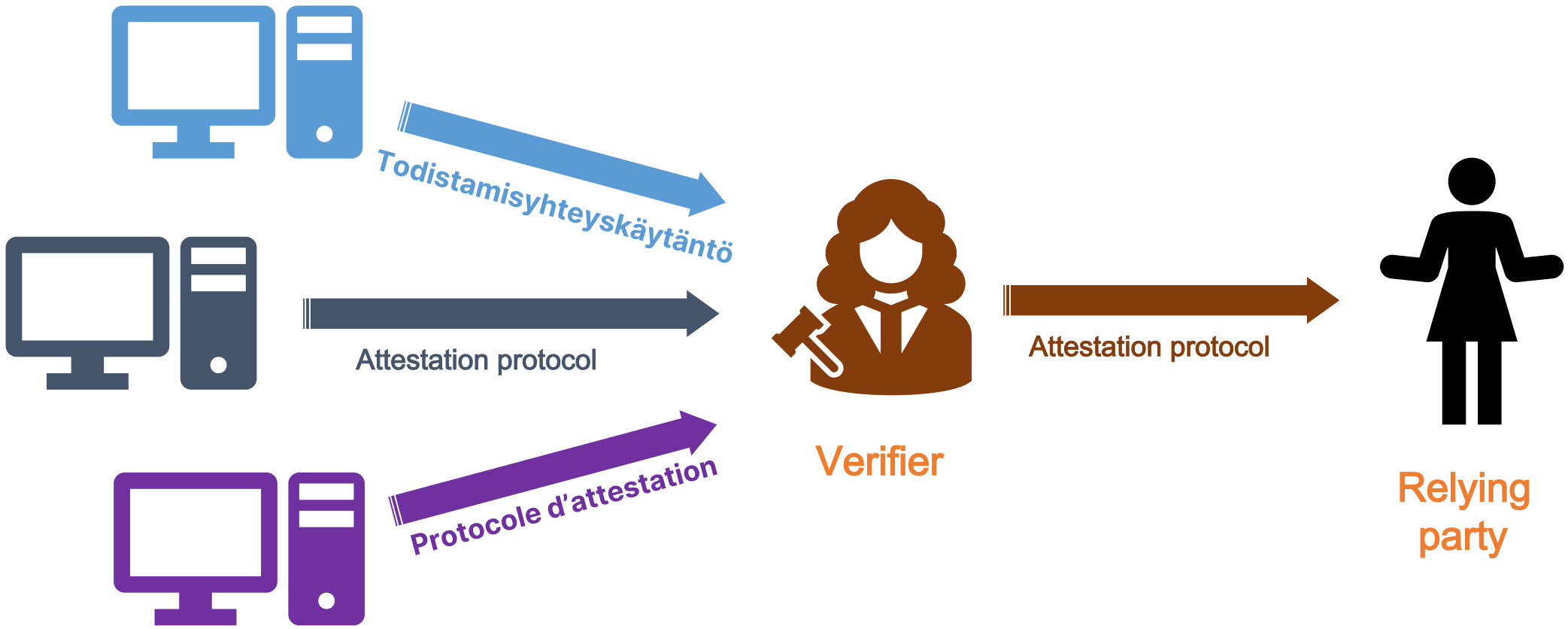
Attester

HW = Samsung A52
OS = Android 11
App = Bank ID

I'm talking to…
HW = Samsung A52
OS = Android 11
App = Bank ID

Co-funded by
The European Union
6GSNS

Verification frameworks



Solution: TrustMee

elacotic

Idea: Attester provides verification code to verifier

- Eliminates all platform-specific code from the verifier

Problem: Malicious verification code can compromise the verifier

Solution: **Protect the verifier using sandboxing techniques from the web**

Problem: How can we trust the verification logic

Solution: **Measure the verification code, now it's just another claim to check**

Co-funded by
The European Union
6GSNS

WebAssembly: An application virtual machine

elacotic

Applications compiled into WebAssembly modules

Modules have (non-exhaustive)

- An isolated memory space
- Exports: functionality exposed to the VMM (e.g. program entry point)
- Imports: functionality requested from the VMM (e.g. storage operations)

Wasm VMs are cheap!

- We can put different parts of an application each in their own sandbox

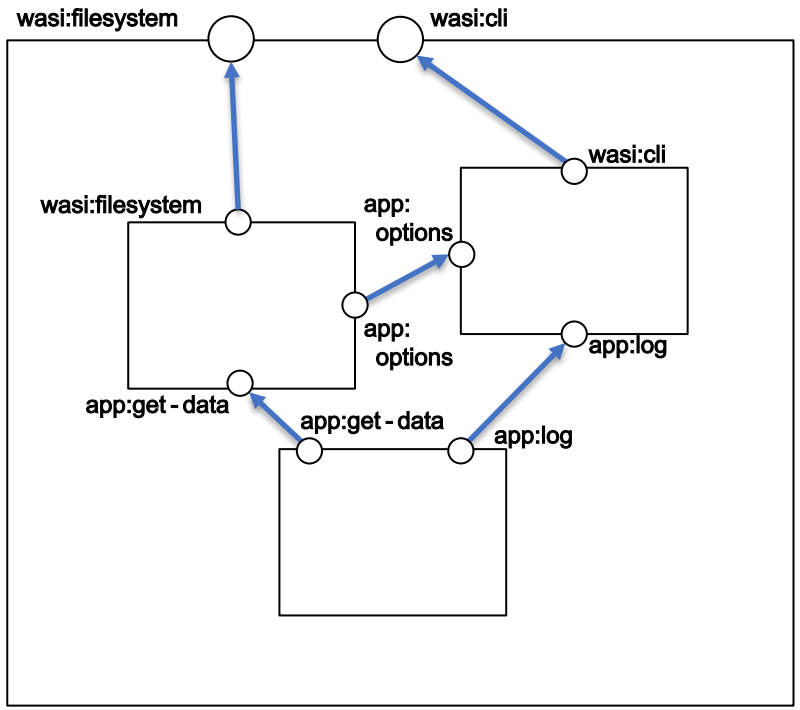
Co-funded by
The European Union
6GSNS

WebAssembly component model

elacotic

Wasm modules can be wrapped into Components with a well-defined ABI

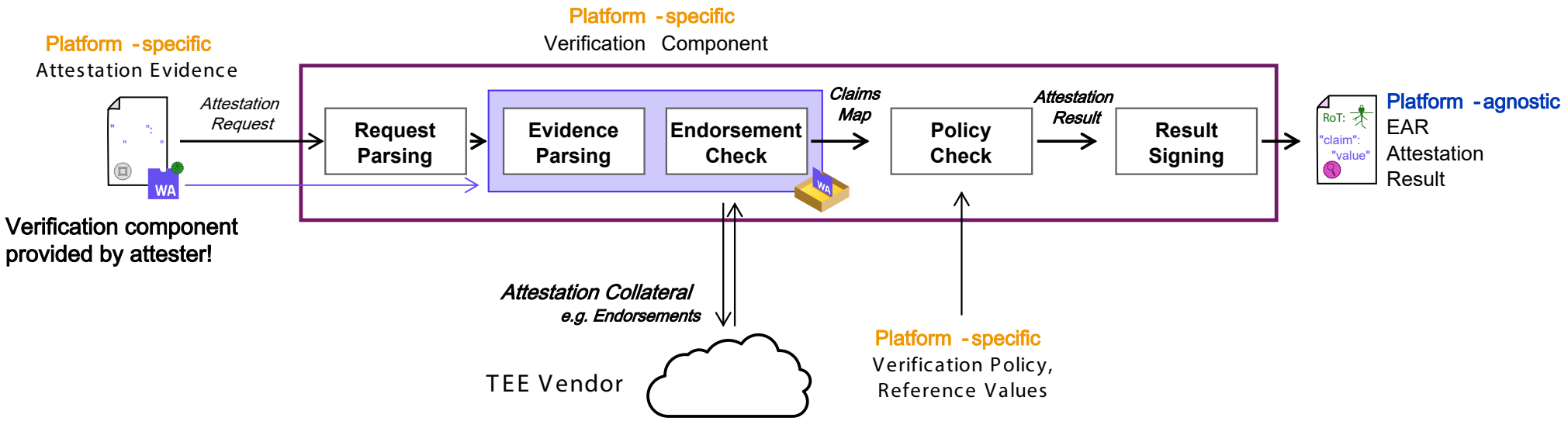
- Wasm components can be composed into larger components




```
package trustee:verifier;

interface verifier-interface {
  variant optional-data {
    value(list<u8>),
    not-provided,
  }

  type report-data = optional-data;
  type init-data-hash = optional-data;

  resource verifier {
    constructor();
    evaluate: func(
      evidence: list<u8>,
      expected-report-data: report-data,
      expected-init-data-hash: init-data-hash,
    ) -> string;
  }
}

world verifier {
  export verifier-interface;
}
```

Co-funded by
The European Union
6GSNS

TrustMee



Co-funded by The European Union

TrustMee implementation

elacontic

Verification component in a Wasm sandbox

- Isolated filesystem region for caching, etc.

Optional network access for OCSP, fetching collateral, etc.

- Only for signed components (for now) to prevent network access from being abused

Implemented as platform driver in Trustee

- EAT+CMW wrapper for platform-specific evidence


```
{
    "tee": "tdx",
    "verifier": "wasm-verification-component",
    "evidence_bundle": {
        "eat_profile": "https://ssg.aalto.fi/trustmee/eat/component-evidence",
        "component_id": "component-<sha256 of-verifier-wasm-bytes>",
        "evidence_type": "application/octet-stream",
        "evidence": "<base64url(raw-evidence-bytes)>"
    }
}
```


Co-funded by The European Union

6GSNS

TrustMee implementation

elacotic

Two implementations, one public

Cross-compatibility tests

Verification components

- AMD SNP
- Intel TDX (Azure)
- AMD SNP again
- Intel TDX (Standalone)

Open-Source

Platform driver implementations

- Non-public prototype

github.com/elasticproject-eu/trustmee-lib

Open-Source

TrustMee implementation

elacotic

Average end-to-end attestation latency for SNP:

	Trustee native	TrustMee*	TrustMee with host crypto**
E2E attestation latency	5.3 ms	14.1 ms	6.5 ms
Platform-specific verification latency	1.1 ms	9.5 ms	2.7 ms

* Warm-start, with compiled verification component cached in memory

** Cryptography operations are performed on the host (in future wasi-crypto should replace it)

Co-funded by
The European Union

6GSNS

The Future

WIP: Eliminating component certification

elacotic

Problem: Verification components isolated, but currently still require certification

- Rogue components could cause DoS (infinite loops, etc.)
- Components needing network access (e.g. for OCSP) potentially risky

Solution: Tighter restrictions still on the component

- Limits on computation time
- Limits on network access for uncertified components, examples:
  - Attester needs to staple collateral
  - Attester needs to proxy the request
  - Grant permission to access domains with special DNS entries
  - Limits on data transfer to prevent reflection attacks

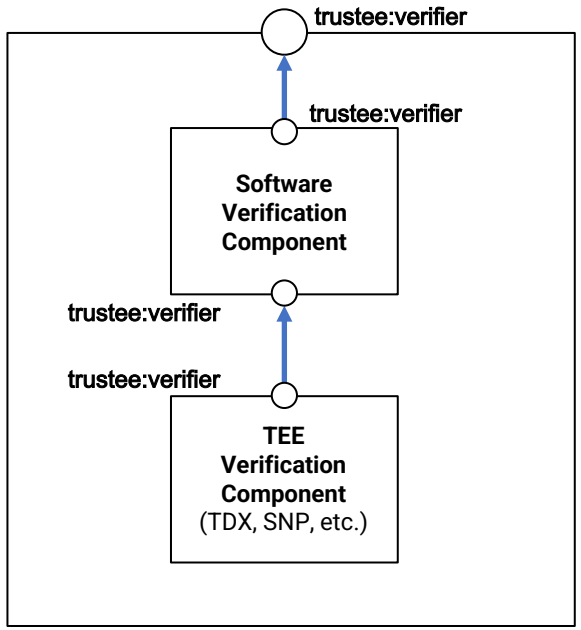
Co-funded by
The European Union
6GSNS

Layered & composite attestation

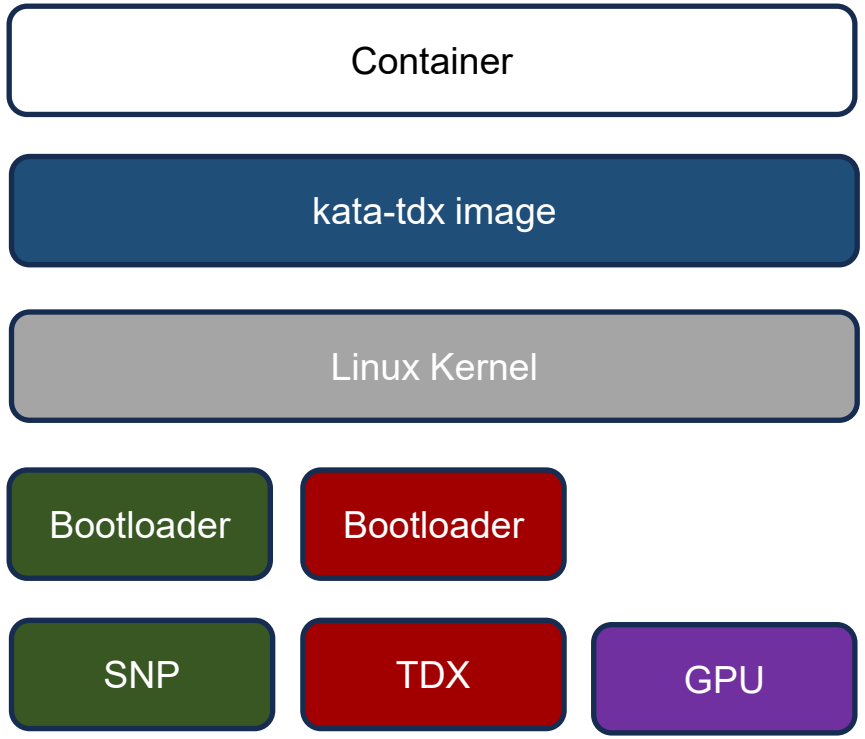
elacotic

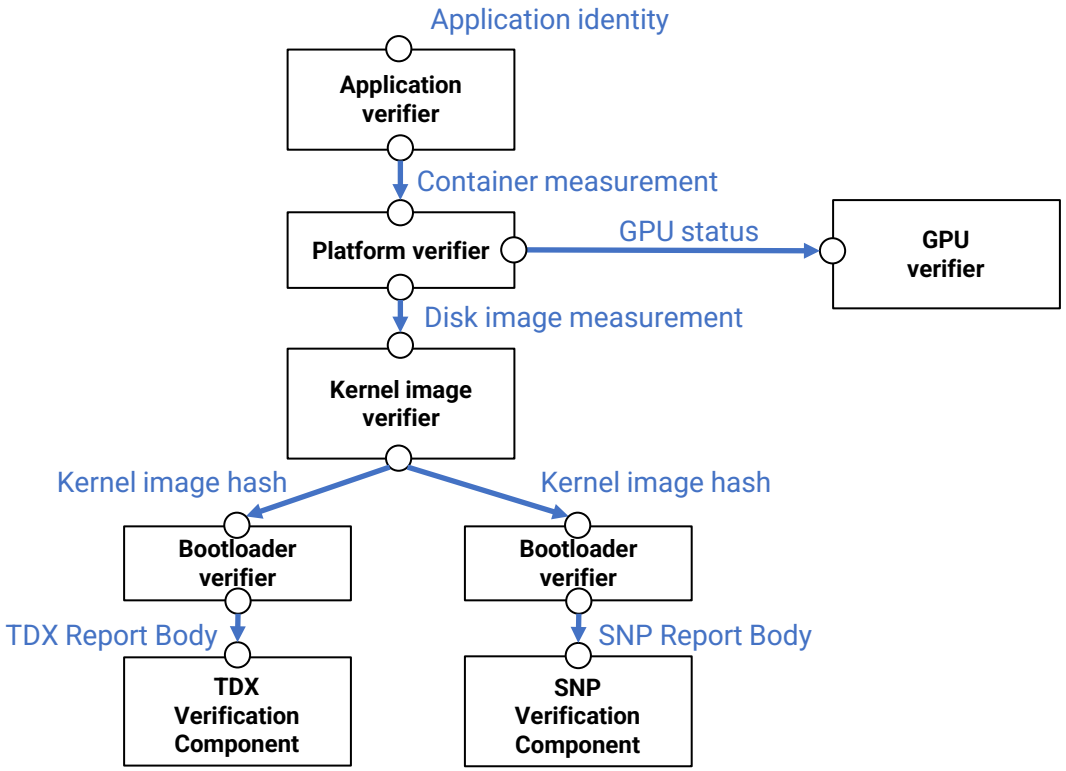
Problem: **Attestation doesn't come just from hardware, but a whole software stack too**

Solution: **Build TrustMee verification components for higher-level application platforms**



Example: container platform



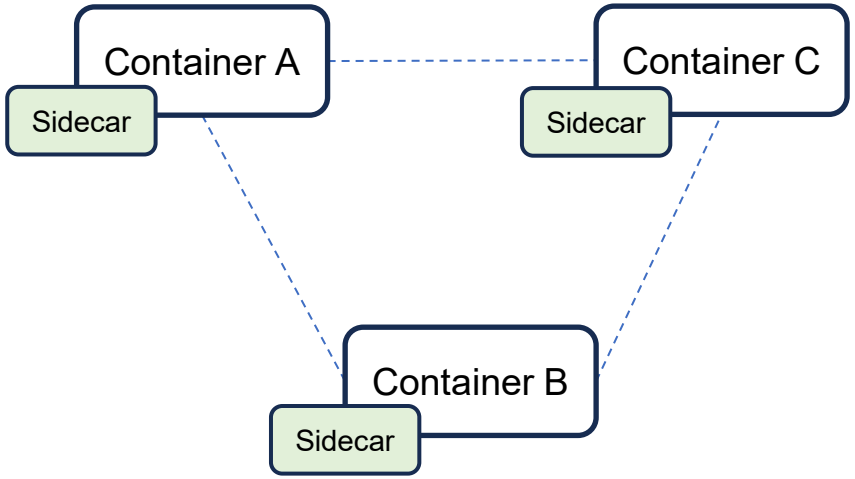


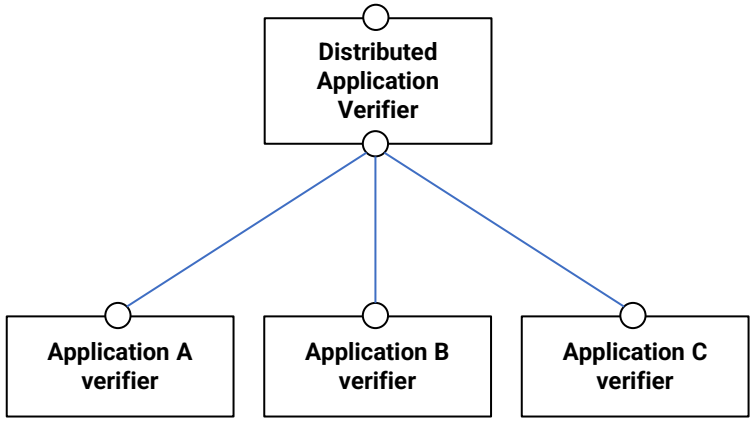
Co-funded by
The European Union

Example: distributed attestation

elacotic

Consensus-like protocol combines component attestations into whole-app attestations





Immediate questions

Encapsulation

elacotic

How should we encapsulate the evidence?

- TrustMee evidence as an EAT token
  - TrustMee-specific EAT profile wrapping platform-specific evidence **Current implementation**
  - Platform-specific EAT profile, TrustMee metadata in JOSE wrapper
    - TrustMee = signature algorithm
    - evidence = signature
    - verification component = public key
    - Allows any EAT profile to self-authenticate without transformation by vendor's attestation service
- TrustMee evidence only as a CMW collection type, don't try to force non-EAT evidence into EAT
  - CMW needed anyway for stapled endorsements

How should we encapsulate the results (as a submod in the EAR document)?

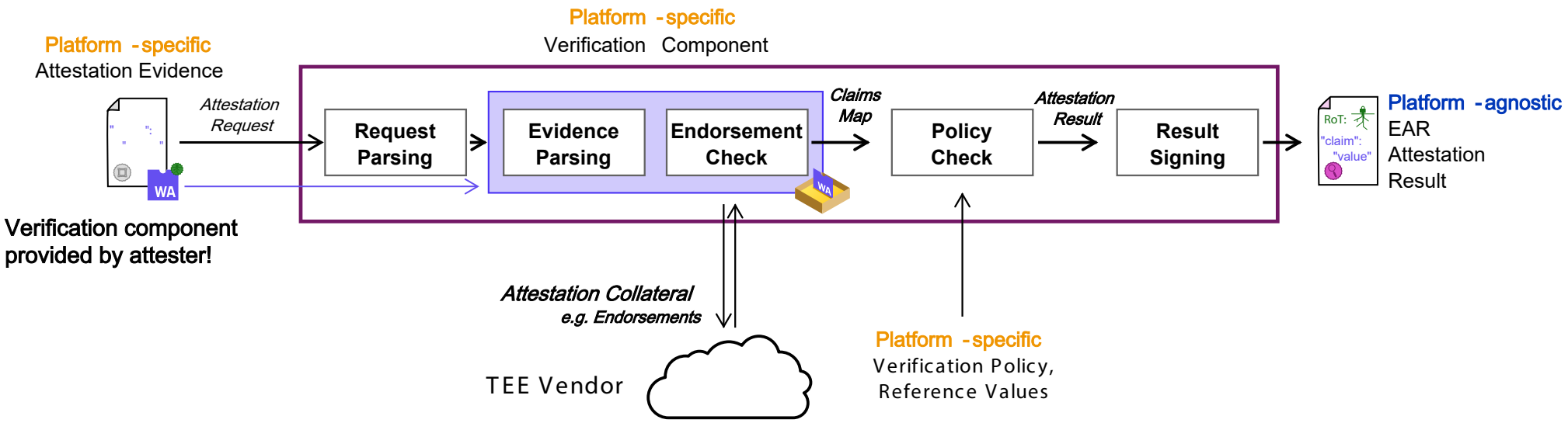
- TrustMee as lead attester, with the verification component producing a nested token
  - Inserts extra layers of nested EAT tokens
  - No profile-compliance issues, since the nested token doesn't have extra claims not in its profile
- TrustMee claims inserted into existing output **Current implementation**
  - Resulting output doesn't comply with any EAT profile

Co-funded by
The European Union
6GSNS

Reference Values

What's the best way to deal with reference values?

- Not currently addressed in TrustMee
- Should verifier components be able to provide or validate reference values themselves?



Summary

TrustMee allows attestation verifiers to eliminate platform-dependent logic

Platform-specific logic can be shared between different verifiers

Prototype available as open-source

- Standalone verification library
- Trustee platform driver

arxiv.org/abs/2602.13148

github .com/elasticproject - eu/trustmee - lib

Co-funded by
The European Union